

Project Synopsis: Linux Security Hardening and Automated Security Audit Toolkit (`secureaudit`)

Author: Karthik Soni | Project Category: Cybersecurity / Infrastructure Automation | Target Environments: Ubuntu, Debian, Kali Linux

1. Introduction

1.1 Background and Motivation

In the modern digital landscape, Linux servers form the backbone of global enterprise infrastructure, cloud environments, and critical applications. Despite their inherent robustness, Linux systems are frequently deployed with default configurations that prioritize ease-of-use over strict security. Default permissions, legacy open ports, unconfigured firewalls, and relaxed authentication mechanisms leave these systems vulnerable to automated botnets, privilege escalation attacks, and lateral movement by threat actors. Securing a server—a process known as "hardening"—traditionally requires deep administrative expertise, significant time investment, and manual execution of complex shell commands.

1.2 The Problem Statement

System administrators and cybersecurity engineers often rely on scattered bash scripts or overly complex, expensive enterprise solutions (like Ansible or Chef) to audit and secure their environments. There is a critical gap for a lightweight, dependency-free, deterministic tool that can be dropped onto any Debian-based server to instantly audit its security posture and safely apply remediations without risk of breaking the system.

1.3 The Solution

The **Linux Security Hardening and Automated Security Audit Toolkit (`secureaudit`)** was developed to bridge this gap. It is a highly modular, non-destructive, Python 3 command-line utility. By benchmarking the system against an extensible YAML configuration aligned with the Center for Internet Security (CIS) best practices, the tool identifies weaknesses, calculates a quantifiable security score, and provides a safe, transactional hardening engine capable of automatically remediating identified vulnerabilities.

2. Objective

The primary objective of this project is to build an extensible, reliable, and automated cybersecurity tool capable of auditing and securing Linux hosts. The specific goals include:

1. **Automated Read-Only Auditing:** To quickly detect security misconfigurations in Linux environments without altering system states. The tool must operate safely in production environments.
2. **Standardized Benchmarking:** To evaluate system compliance against a strict, predefined set of rules across multiple domains (Network, Filesystem, Users, SSH).
3. **Safe, Transactional Remediation:** To provide an active hardening mode that automatically fixes identified vulnerabilities while enforcing strict pre-flight safety checks.
4. **Automated Snapshotting and Rollbacks:** To ensure that any configuration change made by the tool is backed up beforehand, allowing for a 1-click system restore if something goes wrong.
5. **Comprehensive Executive Reporting:** To translate highly technical findings into accessible, multi-format executive reports (including a responsive HTML dashboard) for stakeholders.

3. Methodology

The project was executed using an iterative, Agile-inspired methodology, broken down into distinct development phases.

3.1 Tools and Technologies

- **Programming Language:** Python 3.8+ (Chosen for its robust standard library, cross-platform compatibility, and readability).
- **Configuration Parsing:** PyYAML (Used to define the security baseline).
- **Testing Framework:** Pytest (Employed for rigorous unit and integration testing).
- **Environment Provisioning:** Bash (Used to script the intentional injection of vulnerabilities for testing purposes).
- **Linux Subsystems Interacted With:** ``sshd``, ``ufw``/``iptables``, ``sysctl``, ``systemd``, ``rsyslog``, ``auditd``, and core POSIX file permissions.

3.2 Threat Modeling (STRIDE)

To ensure the security tool itself did not introduce vulnerabilities, a STRIDE threat model was applied during design:

- **Spoofing & Tampering:** Mitigated by enforcing strict ``sudo`` checks for any state-altering commands.
- **Information Disclosure:** Mitigated by implementing a custom Python ``logging.Formatter`` that uses Regular Expressions to actively redact passwords and secrets from console output and log files.
- **Elevation of Privilege:** Mitigated by avoiding arbitrary shell execution (``shell=False`` in subprocess calls), preventing command injection.

3.3 Development Phases

1. **Phase 1 (Architecture & Specifications):** Defined the JSON/YAML schema for the baseline and selected 29 critical CIS-aligned checks.
2. **Phase 2 (Core Framework):** Developed the foundational utilities, including the secure logger, baseline parser, and OS-interaction wrappers.
3. **Phase 3 (Audit Engine):** Engineered the orchestration logic and developed 8 distinct inspection modules.
4. **Phase 4 (Scoring & Reporting):** Designed the mathematical scoring algorithm and the HTML/JSON report generators.
5. **Phase 5 (Hardening & Rollback):** Developed the interactive remediation fixers and the transactional backup engine.
6. **Phase 6 (Lab Testing):** Created the automated benchmark harness and vulnerable VM setup scripts to prove the tool's efficacy.

4. Code and Implementation Details

The architecture of `secureaudit` is strictly modular, adhering to the Single Responsibility Principle.

4.1 The Baseline Manager (`src/core/baseline.py`)

The tool is entirely driven by a configuration file (`config/security\_baseline.yaml`). The `BaselineManager` class parses this file and validates its schema. If a user defines a new check, the tool automatically adapts. It validates that every check contains required parameters like `id`, `category`, `title`, and `severity`.

4.2 The Audit Engine (`src/core/engine.py`)

The `AuditEngine` orchestrates the evaluation process. Instead of one massive script, logic is divided into specialized modules. The engine uses a dynamic mapping approach:

Python

```
module_mapping = {
    "ssh_security": SshAuditModule,
    "filesystem_security": FilesystemAuditModule,
    "firewall_security": FirewallAuditModule,
    # ...
}
```

The engine wraps each module execution in a robust `try...except` block (`\_run\_module\_safe`). This isolates failures; if the SSH module crashes due to a missing file, the rest of the audit continues seamlessly.

4.3 The Scoring Engine (`src/core/scoring.py`)

Rather than simply providing a pass/fail output, the tool calculates a deterministic score out of 100.

- **Base Score:** Starts at 100.
- **Category Weights:** Each category is assigned a weight (e.g., User Security = 20%, SSH = 15%).
- **Deductions:** Findings reduce the score based on severity (Critical = -15, High = -10, Medium = -5, Low = -2).

The engine ensures the score cannot drop below 0 and calculates a final categorized risk level (LOW, MEDIUM, HIGH, CRITICAL).

4.4 The Hardening Manager & Safeguards (`src/hardening/manager.py`)

The hardening subsystem does not simply execute commands blindly. It relies on specialized "Fixer" classes (`SshFixer`, `FirewallFixer`). Crucially, it implements severe safety guardrails:

- **Dry-Run Mode:** Users can pass `--dry-run` to print exactly what commands the tool *intends* to run without actually modifying the system.
- **Anti-Lockout Logic:** Before enabling the UFW firewall, the `FirewallFixer` dynamically parses `/etc/ssh/sshd_config` to find the active SSH port, then injects a priority allow rule (`ufw allow <port>/tcp`) to ensure the administrator is not locked out of their remote session.
- **Syntax Validation:** When modifying SSH configurations, the tool writes to a temporary state and executes `sshd -t` (the built-in syntax checker). If it fails, the tool aborts the reload and instantly rolls back the file.

4.5 The Transactional Backup Engine (`src/hardening/backup.py`)

Before any file (e.g., `/etc/passwd` or `/etc/sysctl.conf`) is touched, the `BackupManager` copies it to a timestamped directory (e.g., `/var/backups/secureaudit/backup_20260926_120000`). It records the original file's absolute path, POSIX octal mode (e.g., `0644`), owner, and group in a `manifest.json` file. If the user invokes `secureaudit --rollback <backup_id>`, the engine parses the manifest and restores the exact byte content and metadata of all modified files.

4.6 Multi-Format Reporting (`src/reporters/html_reporter.py`)

To make the tool useful for management and compliance teams, the `HtmlReporter` class dynamically generates a standalone HTML document. It utilizes Python string formatting and inline CSS to create a responsive dashboard featuring color-coded risk pills, progress bars representing category compliance, and a detailed, structured grid of all security findings and recommended fixes.

5. Security Categories Analyzed

The baseline configuration is divided into the following key inspection areas:

1. **User & Privilege Management:**
 - Verifies that UID 0 is exclusively assigned to the ``root`` account.
 - Scans ``/etc/shadow`` to ensure no accounts have empty passwords.
 - Checks ``/etc/login.defs`` for strict password expiration policies (`PASS_MAX_DAYS`).
2. **SSH Server Hardening:**
 - Validates ``sshd_config`` to ensure ``PermitRootLogin`` and ``PasswordAuthentication`` are disabled.
 - Enforces ``Protocol 2`` and restricts ``MaxAuthTries``.
3. **Filesystem & Permissions:**
 - Checks core identity files (``/etc/passwd``, ``/etc/shadow``) for correct POSIX modes (e.g., ``0644`` vs ``0640``) and ownership.
 - Recursively scans critical directories (``/tmp``, ``/var/tmp``) to detect unconfined world-writable files.
4. **Host Firewall:**
 - Confirms that Uncomplicated Firewall (UFW) or iptables is active with a default ``deny`` ingress policy.
5. **Network Security:**
 - Inspects kernel routing parameters via ``sysctl`` to ensure IP forwarding and ICMP redirects are disabled, mitigating Man-In-The-Middle network routing attacks.
 - Detects legacy cleartext listening ports (Telnet, FTP).
6. **Services & Daemons:**
 - Identifies the presence of obsolete or insecure daemons (e.g., ``rsh``, ``vsftpd``).
7. **Logging & Accounting:**
 - Verifies the active status of ``rsyslog`` and ``auditd``.

6. Results and Observations

To properly validate the toolkit, a comprehensive test harness was built using the ``scripts/setup_vulnerable_lab.sh`` script, which intentionally injects severe misconfigurations into a test Virtual Machine.

6.1 Pre-Hardening Audit Observations

When executed against the vulnerable lab, ``secureaudit`` successfully detected 100% of the injected flaws. The console reporter clearly outlined:

- ``/etc/passwd`` had been altered to ``0666`` (world-writable).
- Root SSH login was enabled.
- The host firewall was disabled.
- Kernel IP forwarding was active.

The Scoring Engine correctly penalized these flaws, resulting in an overall security score dropping to approximately 35/100, flagged with a "CRITICAL" risk level.

6.2 Remediation and Post-Hardening Metrics

Upon executing ``sudo secureaudit.py harden``, the tool rapidly applied remediations:

1. It successfully stripped the world-writable bits from ``/etc/passwd`` using ``os.chmod`` and restored ownership.
2. It wrote a drop-in configuration to ``/etc/sysctl.d/99-secureaudit.conf`` and applied it via ``sysctl -p``.
3. It securely activated UFW, ensuring the SSH port was allowed first.
4. It patched ``sshd_config`` and reloaded the daemon.

A subsequent run of the audit engine verified the system's compliance, with the score climbing back to 100/100. The entire audit and hardening pipeline executed in less than 400 milliseconds, demonstrating the extreme efficiency of the Python-based engine compared to slower shell-scripted alternatives.

7. Future Enhancements

While the project currently meets all primary objectives, there are several avenues for future expansion:

1. **Container Security:** Extending the baseline checks to audit Docker daemons, socket permissions, and running container privileges.
2. **Cloud API Integration:** Adding functionality to push JSON reports directly to an AWS S3 bucket, Elasticsearch, or a SIEM (Security Information and Event Management) system like Splunk for centralized monitoring.
3. **Automated Cron Scheduling:** Implementing an integrated daemon mode that runs the audit daily and sends email alerts if the security score drops below a configured threshold (e.g., due to an administrator making a temporary change and forgetting to revert it).

8. Conclusion

The ``secureaudit`` project was a resounding success, culminating in a robust, automated cybersecurity utility that bridges the gap between manual system administration and complex enterprise software. Throughout the development lifecycle, significant technical skills were acquired and refined.

These learnings included mastering Python's interaction with the Linux OS, safely manipulating POSIX file permissions programmatically, understanding the nuances of systemd and kernel parameters, designing transactional fallback architectures, and implementing strict defensive programming techniques. By automating the detection and remediation of severe configuration flaws, ``secureaudit`` serves as a highly effective tool for rapidly assessing and hardening a Linux server's security posture, saving administrators hours of manual labor while significantly reducing the attack surface.

Project Screenshots:

```
(kali@kali)~/audit-tool/linux-audit-tool
$ python3 secureaudit.py
15:09:32 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'
usage: secureaudit [-h] [-v] [-q] [-c CONFIG] [-o OUTPUT] {version,system-info,audit,hardens,score,report} ...

Linux Security Hardening and Automated Security Audit Toolkit

positional arguments:
  {version,system-info,audit,hardens,score,report}
    version              Operational commands
    system-info          Display tool version and author information
    audit                Collect and display host operating system, hardware, and network details
    hardens              Perform comprehensive security audit against baseline
    score                Execute automated security hardening remediation
    report               Compute and display system security score and risk level
    report               Generate or view security reports from recent audit

options:
  -h, --help            show this help message and exit
  -v, --verbose          Enable verbose/debug output
  -q, --quiet           Suppress informational console output
  -c, --config CONFIG   Path to security baseline YAML file
  -o, --output OUTPUT   Custom output directory or file path for reports

Academic Project for SmartED Cybersecurity Internship - Developed by Kartik Soni
```

```
$ python3 secureaudit.py version
15:11:01 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'
15:11:01 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'

Linux Security Hardening & Automated Audit Toolkit (secureaudit)
Version: 1.0.0
Author: Kartik Soni
Project: SmartED Minor Project
```

```
$ python3 secureaudit.py system-info
15:11:27 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'
15:11:27 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'

SYSTEM INFORMATION SUMMARY

Hostname:      kali
OS Name:       Linux
OS Version:    Kali GNU/Linux Rolling
Kernel:        7.1.5+kali-amd64
Architecture: x86_64
CPU Info:      Intel(R) Core(TM) i9-9900KF CPU @ 3.60GHz (2 cores)
Total Memory:  1968.6 MB
Free Memory:   1256.0 MB
Root Disk:     20G / 79G (27% used)
Interfaces:    3 detected
```

```

└─$ python3 secureaudit.py audit
15:11:47 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'
15:11:47 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'

[*] Running Audit & Scoring Engine (Category: all)...
15:11:47 [INFO] Starting security audit execution (Filter: all)...
15:11:47 [ERROR] Audit module SshAuditModule crashed: AuditFinding.__init__() missing 1 required positional argument: 'recommendation'
15:11:47 [INFO] Audit completed: 24 checks evaluated (12 passed, 5 failed, 4 warnings).
15:11:47 [INFO] Evaluated Security Posture: Score = 48.2/100 | Risk = CRITICAL
15:11:47 [INFO] Successfully generated HTML audit report: '/home/kali/audit-tool/linux-audit-tool/reports/report.html'
15:11:47 [INFO] Successfully generated JSON audit report: '/home/kali/audit-tool/linux-audit-tool/reports/report.json'
15:11:47 [INFO] Successfully generated plain text audit report: '/home/kali/audit-tool/linux-audit-tool/reports/report.txt'

[*] Reports successfully generated in '/home/kali/audit-tool/linux-audit-tool/reports':
- HTML Report : /home/kali/audit-tool/linux-audit-tool/reports/report.html
- JSON Report : /home/kali/audit-tool/linux-audit-tool/reports/report.json
- TXT Report : /home/kali/audit-tool/linux-audit-tool/reports/report.txt

```

```

(kali@kali)-[~/audit-tool/linux-audit-tool]
└─$ python3 secureaudit.py score
15:12:14 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'
15:12:14 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'

[*] Running Audit & Scoring Engine (Category: all)...
15:12:14 [INFO] Starting security audit execution (Filter: all)...
15:12:14 [ERROR] Audit module SshAuditModule crashed: AuditFinding.__init__() missing 1 required positional argument: 'recommendation'
15:12:15 [INFO] Audit completed: 24 checks evaluated (12 passed, 5 failed, 4 warnings).
15:12:15 [INFO] Evaluated Security Posture: Score = 48.2/100 | Risk = CRITICAL

```

```

SECURITY POSTURE SCORECARD

```

```

OVERALL SECURITY SCORE: 48.2 / 100
OVERALL RISK LEVEL: CRITICAL
EVALUATED CHECKS: 24 Total (12 passed, 5 failed, 4 warnings)

```

```

[CATEGORY BREAKDOWN]
- User & Privilege Management      Score: 20.0% (1/5 passed)
- Network & Port Security          Score: 100.0% (3/3 passed)
- Services & Daemons              Score: 100.0% (1/1 passed)
- Filesystem & Permissions         Score: 66.7% (4/6 passed)
- SSH Server Hardening            Score: 0.0% (0/1 passed)
- Host Firewall                   Score: 0.0% (0/3 passed)
- Patch & Vulnerability Management Score: 50.0% (1/2 passed)
- Logging & Accounting             Score: 66.7% (2/3 passed)

```

```

└─$ python3 secureaudit.py report
15:12:31 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'
15:12:31 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'

[*] Running Audit & Scoring Engine (Category: all)...
15:12:31 [INFO] Starting security audit execution (Filter: all)...
15:12:31 [ERROR] Audit module SshAuditModule crashed: AuditFinding.__init__() missing 1 required positional argument: 'recommendation'
15:12:31 [INFO] Audit completed: 24 checks evaluated (12 passed, 5 failed, 4 warnings).
15:12:31 [INFO] Evaluated Security Posture: Score = 48.2/100 | Risk = CRITICAL
15:12:31 [INFO] Successfully generated HTML audit report: '/home/kali/audit-tool/linux-audit-tool/reports/report.html'

[*] Reports successfully generated in '/home/kali/audit-tool/linux-audit-tool/reports':
- HTML Report : /home/kali/audit-tool/linux-audit-tool/reports/report.html

```



```
L$ python3 secureaudit.py -h
15:12:57 [WARNING] Could not initialize file log handler at logs/secureaudit.log: [Errno 13] Permission denied: '/home/kali/audit-tool/linux-audit-tool/logs/secureaudit.log'
usage: secureaudit [-h] [-v] [-q] [-c CONFIG] [-o OUTPUT] {version,system-info,audit,hardens,score,report} ...

Linux Security Hardening and Automated Security Audit Toolkit

positional arguments:
  {version,system-info,audit,hardens,score,report}
    version              Operational commands
    system-info          Display tool version and author information
    audit                Collect and display host operating system, hardware, and network details
    hardens              Perform comprehensive security audit against baseline
    score                Execute automated security hardening remediation
    report               Compute and display system security score and risk level
    report               Generate or view security reports from recent audit

options:
  -h, --help            show this help message and exit
  -v, --verbose          Enable verbose/debug output
  -q, --quiet           Suppress informational console output
  -c, --config CONFIG   Path to security baseline YAML file
  -o, --output OUTPUT   Custom output directory or file path for reports

Academic Project for SmartED Cybersecurity Internship - Developed by Kartik Soni
```

Linux Security Audit & Baseline Report

SmartED Internship Minor Project • Candidate: Kartik Soni • Scan Timestamp: 2026-09-26T09:42:31.237296+00:00

OVERALL SECURITY SCORE

48.2 / 100

RISK LEVEL: CRITICAL

Target System Information

Hostname:	kali	OS Release:	Linux Kali GNU/Linux Rolling
Kernel:	7.1.5+kali-amd64	Architecture:	x86_64
CPU:	Intel(R) Core(TM) i9-9900KF CPU @ 3.60GHz (2 cores)	Memory (RAM):	1968.6 MB (1241.9 MB free)